

A long-form mathematical primer for HSiKAN, Gömb-strict, GömbSoma, Sequential HSiKAN, and the σ -leakage audit framework

Internal companion to the Nature Communications submission

2026-05-17

Abstract

This primer is a single-document refresher on every piece of mathematics used by HSiKAN, Gömb-strict (signed-graph link prediction), GömbSoma (vision), Sequential HSiKAN + CliffordFIR (sequences), and the σ -leakage audit framework. It is written to be read end-to-end in one or two evenings: the definitions, the small number of theorems we actually rely on, the proofs where they are short enough to clarify rather than distract, and worked examples on the canonical datasets (Bitcoin Alpha, Slashdot, Epinions, Cluttered MNIST).

Each section closes with a “codebase” pointer naming the file that implements the construction so you can cross-reference math and code without leaving the document.

Contents

I	Foundations of signed and hypergraph structure	1
1	Signed graphs, balance, and frustration	1
1.1	Definitions	1
1.2	Switching equivalence	2
1.3	Balance theory	2
1.4	Frustration	3
1.5	Spectral structure: the signed Laplacian	4
1.6	Codebase pointers	4
2	Cycles: enumeration, scoring, and pooling	4
2.1	Cycle enumeration is hard	4
2.2	Branch-and-bound enumeration with bounded scores	5
2.3	Scoring functions	5
2.4	σ -cycle pooling	5
2.5	Mixed-arity pooling and the α_k simplex	6
2.6	Walk-vs-cycle: when walks add value	6
2.7	Codebase pointers	6

II	Function approximation: KAN and HSiKAN	6
3	The Kolmogorov-Arnold representation	6
3.1	The classical theorem	6
3.2	From the theorem to the KAN architecture	7
3.3	Univariate basis: B-splines vs Catmull-Rom	7
3.4	The HSiKAN aggregator	8
3.4.1	Expressiveness	8
3.4.2	Why σ -masking changes the function class	9
3.5	Codebase pointers	9
III	Hypergraph cycle pooling: the Gömb cascade	9
4	Signed hypergraphs	10
4.1	Definitions	10
4.2	Cycle-as-hyperedge	10
4.3	Hypergraph Laplacians	10
4.4	Codebase pointers	10
5	The Gömb three-shell cascade	11
5.1	Architecture overview	11
5.2	Tier organisation: hierarchical n_{tiers}	11
5.3	The CPML topology variants	12
5.4	Capsule routing (optional)	12
5.5	Codebase pointers	12
IV	Discrete differential geometry on graphs and cell complexes	12
6	Cell complexes and boundary operators	12
6.1	Definitions	12
6.2	Hodge Laplacian and decomposition	13
6.3	Forman-Ricci curvature	14
6.4	Ollivier-Ricci as an alternative	14
6.5	Codebase pointers	15
7	The Bochner-Hodge hypergraph convolution	15
7.1	Bochner-style operator	15
7.2	Codebase pointers	15

V	Clifford algebras and the Sequential dual-path architecture	15
8	Clifford algebras	16
8.1	The Clifford algebra $\text{Cl}(p, q)$	16
8.2	Small Clifford algebras: complete table	16
8.3	$\text{Cl}(2, 0)$ in detail	16
8.4	Inner / outer / geometric product decomposition	17
8.5	Reflections and rotations	17
8.6	Codebase pointers	18
9	CliffordFIR and the Sequential HSiKAN dual path	18
9.1	Causal multivector FIR	18
9.2	Multi-channel extension	18
9.3	Sequential HSiKAN: windowed σ -cycle	18
9.4	The dual-path block	18
9.5	Load-balance regularisation	19
9.6	Codebase pointers	19
VI	Methodology: leakage, audit, and statistical protocol	20
10	The transductive convention and the leakage mechanism	20
10.1	Setup: signed link prediction	20
10.2	Two protocols	20
10.3	The leakage mechanism, formal	20
10.4	Empirical leakage on Bitcoin Alpha	21
10.5	Codebase pointers	21
11	The label-shuffle audit	21
11.1	Procedure	21
11.2	Theoretical justification	21
11.3	Measured results, Bitcoin Alpha	22
12	Statistical protocol: paired-by-seed comparison	22
12.1	Setup	22
12.2	The paired z -score	22
12.3	Why paired beats unpaired	23
12.4	Power analysis	23
12.5	Codebase pointers	23

13 Metrics: accuracy, F1, AUC, and their relations	23
13.1 Definitions	23
13.2 Accuracy recovery from per-class P/R	24
13.3 Class-imbalance and the metric trap	24
13.4 Codebase pointers	24
 VII Reference material	 24
 14 Glossary	 24
 15 Notation index	 26
 16 Further reading	 27
 17 Codebase quick reference: every concept, every file	 29
 18 Open mathematical questions	 31

Part I

Foundations of signed and hypergraph structure

1 Signed graphs, balance, and frustration

1.1 Definitions

Definition 1.1 (Signed graph). *A signed graph is a triple $G = (V, E, \sigma)$ where*

- *V is a finite vertex set with $|V| = n$;*
- *$E \subseteq \binom{V}{2}$ is an undirected edge set (or $E \subseteq V \times V$ if directed) with $|E| = m$;*
- *$\sigma : E \rightarrow \{-1, +1\}$ is a signature.*

The underlying graph is $G^\pm = (V, E)$, obtained by forgetting the signature. We write $E^+ = \sigma^{-1}(+1)$ and $E^- = \sigma^{-1}(-1)$.

Definition 1.2 (Walk, path, cycle). *A walk of length k is an alternating sequence $v_0, e_1, v_1, e_2, \dots, e_k, v_k$ with $e_i = (v_{i-1}, v_i)$; a path is a walk with all v_i distinct; a cycle is a closed walk $v_0 = v_k$ with $v_0, v_1, \dots, v_{k-1}$ distinct.*

Definition 1.3 (Cycle sign / σ -product). *For a cycle $c = (v_0, v_1, \dots, v_{k-1}, v_0)$ in a signed graph G , the cycle sign (or σ -product) is*

$$\sigma(c) := \prod_{i=0}^{k-1} \sigma(v_i, v_{i+1 \bmod k}) \in \{-1, +1\}.$$

The cycle sign is the only signed invariant of a closed walk that is permutation- and direction-invariant. In a signed graph, **the structural data lives in the σ -products of cycles**; anything not derivable from these is essentially an unsigned-graph property.

1.2 Switching equivalence

Definition 1.4 (Switching). *For a vertex subset $S \subseteq V$, the switch at S of a signed graph $G = (V, E, \sigma)$ is the new signed graph $G^S = (V, E, \sigma^S)$ with*

$$\sigma^S(e) = \begin{cases} -\sigma(e) & \text{if } e \text{ has exactly one endpoint in } S, \\ +\sigma(e) & \text{otherwise.} \end{cases}$$

Two signed graphs G_1 and G_2 on the same underlying graph are switching-equivalent if $G_2 = G_1^S$ for some $S \subseteq V$.

Lemma 1.5 (Zaslavsky 1982). *Switching preserves σ -cycle signs: if $G_2 = G_1^S$ then for every cycle c , $\sigma_{G_1}(c) = \sigma_{G_2}(c)$.*

Proof sketch. Each edge with both endpoints in S keeps its sign; each edge with both endpoints outside S keeps its sign; only edges with exactly one endpoint in S are negated. A cycle traversing the boundary between S and $V \setminus S$ crosses an even number of such edges (each traversal in must be matched by a traversal out), so the product of negations is $(-1)^{\text{even}} = +1$. $\square$

Intuition. Switching is the signed-graph analogue of changing the sign convention on a basis. If we negate the entire “community A” side of a network, every σ -product is unchanged: positive edges between two community-A vertices stay positive (both negated, $(-1)(-1) = +1$ on the path through them); but a single crossing edge (one endpoint each in A and $\neg A$) gets negated.

1.3 Balance theory

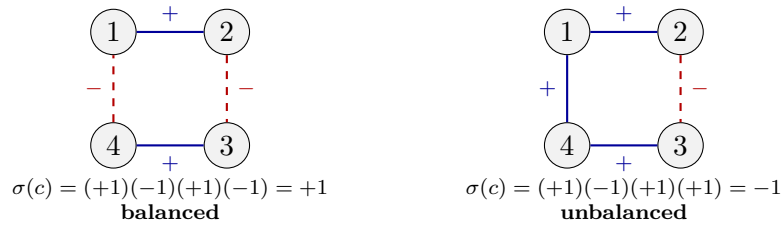


Figure 1: **Balanced vs unbalanced 4-cycles.** The signature $\sigma : E \rightarrow \{-1, +1\}$ is the colour of each edge (blue solid = $+1$, red dashed = -1). A cycle is balanced iff its sign product is $+1$. The left graph has an even number of negative edges; the right has an odd number.

Definition 1.6 (Balanced graph). *A signed graph is balanced if $\sigma(c) = +1$ for every cycle $c \in G$.*

Theorem 1.7 (Cartwright & Harary 1956; Harary 1953). *The following are equivalent for a connected signed graph $G = (V, E, \sigma)$:*

- (i) Every cycle in G has $\sigma(c) = +1$.
- (ii) There exists a partition $V = V^+ \sqcup V^-$ such that all edges within V^+ and within V^- are positive, and all edges between V^+ and V^- are negative.
- (iii) G is switching-equivalent to the all-positive graph $(V, E, +1)$.

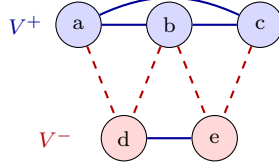


Figure 2: **Cartwright–Harary 2-partition.** Theorem 1.7 guarantees that a balanced signed graph G admits a partition $V = V^+ \sqcup V^-$ such that within-partition edges are positive (blue) and between-partition edges are negative (red dashed). The two “sides” are the social communities; every closed walk that crosses between them does so an even number of times, giving $\sigma(c) = +1$.

Proof sketch. (i) $\Rightarrow$ (ii): Pick a root vertex v_0 and put $v_0 \in V^+$. For each other vertex v , choose a path P from v_0 to v and put $v \in V^+$ if $\sigma(P) = +1$ and $v \in V^-$ otherwise. Independence of path choice: if P, P' are two v_0 -to- v paths, then $P \cup P'^{-1}$ is a closed walk (possibly non-simple); by hypothesis its sign is $+1$, so $\sigma(P) = \sigma(P')$, well-defining the partition. Verify the sign condition on each edge directly.

(ii) $\Rightarrow$ (iii): Switch at V^- . Edges within V^- stay positive (both endpoints negated, $(-)(-) = +$); cross edges are negated $-1 \rightarrow +1$; within- V^+ edges are unchanged. Result is all-positive.

(iii) $\Rightarrow$ (i): Lemma 1.5: switching preserves cycle signs; the all-positive graph has all cycle signs $+1$; therefore G does too. $\square$

1.4 Frustration

Real-world networks are rarely balanced; we measure their *frustration* as a quantitative departure from balance.

Definition 1.8 (Frustration index). *Given a signed graph G , the (edge) frustration index is*

$$L(G) = \min_{S \subseteq V} |\{e \in E : \sigma^S(e) = -1\}|,$$

the minimum number of negative edges after any switching. Equivalently, $L(G)$ is the minimum number of edges to delete from G to make it balanced.

Computing $L(G)$ exactly is NP-hard (reduction from MAX-CUT), but several proxies are tractable:

- *Cycle frustration:* the fraction of cycles in some reference pool $\mathcal{C}$ with $\sigma(c) = -1$. We compute this approximately by sampling cycles in `hymeko_py/src/cycles/topk.rs`.
- *Triadic frustration:* the cycle-frustration measure restricted to triangles, computable in $O(m^{3/2})$ time via triangle listing.
- *Spectral frustration:* bounded by the smallest eigenvalue of the signed Laplacian (see §1.5).

Empirical values on benchmark datasets:

Dataset	$ V $	approx. triadic frustration
Bitcoin Alpha	3 783	7%
Bitcoin OTC	5 881	9%
Slashdot	82 140	23%
Epinions	131 580	15%

1.5 Spectral structure: the signed Laplacian

Definition 1.9 (Signed adjacency and Laplacian). *For a signed graph G , the signed adjacency matrix $A_\sigma \in \mathbb{R}^{n \times n}$ has entries $(A_\sigma)_{uv} = \sigma(u, v)$ if $(u, v) \in E$, 0 otherwise. The signed degree matrix $\bar{D} = \text{diag}(\bar{d})$ has $\bar{d}_u = \sum_v |A_\sigma|_{uv}$ (the unsigned degree). The signed Laplacian is*

$$L_\sigma = \bar{D} - A_\sigma.$$

Note the use of unsigned degrees in $\bar{D}$: this makes L_σ *positive semidefinite* (unlike the naive $D - A_\sigma$ with signed degrees, which can have negative eigenvalues for unbalanced graphs).

Proposition 1.10 (Kunegis et al. 2010). *For a connected signed graph G , the smallest eigenvalue $\lambda_1(L_\sigma) = 0$ if and only if G is balanced. Otherwise $\lambda_1(L_\sigma) > 0$, and the gap quantifies distance-to-balance.*

Proof sketch. $\lambda_1 \geq 0$ by Gershgorin (every row of L_σ has diagonal $\bar{d}_u$ and off-diagonal absolute-sum $\bar{d}_u$). For the iff: $\lambda_1 = 0$ with eigenvector f means $\bar{d}_u f_u = \sum_v A_\sigma(u, v) f_v$ for every u ; rearranging, $\sigma(u, v) f_u f_v = f_v^2$ (after multiplying both sides by f_v and re-summing), which on a connected graph forces $|f_u| = |f_v|$ for all u, v and $\sigma(u, v) = \text{sgn}(f_u f_v)$. This is exactly the balance partition. $\square$

1.6 Codebase pointers

Signed-graph data class	<code>signedkan_wip/src/datasets.py:SignedGraph</code>
Bitcoin / Slashdot / Epinions loader	<code>signedkan_wip/src/datasets.py:load</code>
Reddit Hyperlinks loader (Phase C)	<code>datasets.py:load("reddit_body" / "reddit_title")</code>

2 Cycles: enumeration, scoring, and pooling

2.1 Cycle enumeration is hard

Listing all cycles of length exactly k in a graph is #P-hard in general; even just counting 4-cycles is intractable for dense graphs. The standard tractable strategies are:

- (I) *Full enumeration with hard cap* for small graphs and small k . Bitcoin Alpha at $k = 5$: 171 358 cycles, fits in memory. Slashdot at $k = 5$: > 55 million, doesn't.
- (II) *Color-coding* (Alon, Yuster, Zwick 1995): random k -coloring of V ; cycles are enumerated only if *colourful* (all vertices distinct colors). Repeat $O(\binom{k}{k/2})$ times for a constant-factor approximation.
- (III) *Branch-and-bound by per-vertex score* with a sum-of- sub-path-bounds upper-bound function (see §2.2).
- (IV) *Walk closure*: enumerate length- $(k - 1)$ walks, check if the missing edge closes the walk into a cycle.

2.2 Branch-and-bound enumeration with bounded scores

For top- m -per-vertex cycle selection with score function $s : \mathcal{C} \rightarrow \mathbb{R}_{\geq 0}$, we maintain a per-vertex heap of size m during DFS. The DFS explores a tree of partial paths $P = (v_0, v_1, \dots, v_\ell)$ with $\ell < k$. We need an *upper bound* on the maximum score of any cycle extending P .

Definition 2.1 (Bounded scorer). *A score function s is bounded if there exists $\bar{s} : (\text{partial path}) \rightarrow \mathbb{R}_{\geq 0}$ such that for every partial path P and every cycle c extending P ,*

$$s(c) \leq \bar{s}(P).$$

If the heap is already full of cycles of score $\geq s^*$, we can prune the DFS branch at P whenever $\bar{s}(P) \leq s^*$. This is the core speedup behind the ABB enumerator (`hymeko_py/src/cycles/topk.rs`): on Epinions at $k = 5$, ABB takes 4.0 s for $K = 10\text{k}$ cycles versus 100.5 s for the non-bounded enumerator.

2.3 Scoring functions

Balance fraction $s_{\text{bal}}(c) = \mathbf{1}[\sigma(c) = +1]$. Counts balanced cycles. Bounded: a partial path either already contains a $\sigma = -1$ edge (so the final cycle sign is at best -1) or doesn't (final at best $+1$).

Davis index $s_{\text{Dav}}(c) = \mathbf{1}[c \text{ has all-positive or all-negative or mixed-balanced pattern}]$. Generalises balance to triadic patterns.

Unbalanced fraction $s_{\text{unbal}}(c) = \mathbf{1}[\sigma(c) = -1]$. Counts unbalanced cycles — the dual of s_{bal} .

Weighted sum $s_{\text{w}}(c) = \sum_i w_i \cdot \text{feature}_i(c)$ for arbitrary feature combinations.

The ABB extension of these scorers tracks both an upper and lower bound on the cycle score as the DFS extends, allowing branch pruning when the upper bound falls below the heap floor.

2.4 σ -cycle pooling

Definition 2.2 (Cycle pool of arity k). $\mathcal{C}_k(G) := \{c : c \text{ is a } k\text{-cycle in } G\}$, with each cycle counted once up to rotation and reflection.

Definition 2.3 (σ -weighted cycle pool feature). *Given per-vertex embeddings $h_v \in \mathbb{R}^d$ and a cycle $c = (v_0, \dots, v_{k-1}, v_0)$, define a per-cycle feature $\phi(c) = \text{Aggregate}(h_{v_0}, \dots, h_{v_{k-1}})$ via some permutation-invariant aggregator (mean, max, sum). The σ -weighted cycle pool feature for vertex v is*

$$z_v^{(k)} := \sum_{c \in \mathcal{C}_k(G) : v \in c} \sigma(c) \cdot \phi(c).$$

Why σ -weighting matters. Without the $\sigma(c)$ factor, $z_v^{(k)}$ depends only on the underlying graph and the per-vertex embeddings — the signature σ is forgotten. The σ -weighted variant is the simplest aggregator that uses cycle-level signed structure.

2.5 Mixed-arity pooling and the α_k simplex

For $k \in \mathcal{K} \subseteq \{3, 4, 5, 6\}$, we compute $z_v^{(k)}$ for each arity, then mix:

$$z_v = \sum_{k \in \mathcal{K}} \alpha_k z_v^{(k)}, \quad \alpha = \text{softmax}(\theta).$$

θ is learned jointly with the rest of the network. The learned distribution α acts as a *regime compass*:

Dataset	α_3	α_4	α_5	dominant
Bitcoin Alpha	~ 0.04	~ 0.45	~ 0.51	4 + 5
Bitcoin OTC	~ 0.06	~ 0.49	~ 0.45	4 + 5
Slashdot	~ 0.18	~ 0.36	~ 0.46	5
Epinions	~ 0.22	~ 0.39	~ 0.39	4 + 5
SBM $k = 4$ (synthetic)	~ 0.55	~ 0.40	~ 0.05	3

2.6 Walk-vs-cycle: when walks add value

A cycle of length k contains $\binom{k}{2}$ shorter walks. A *walk-only aggregator* (used in the Walk-HSiKAN variant) sums over open walks instead of closed cycles, with the walk’s product of edge signs replacing the cycle’s σ . Walks help in two regimes:

1. Very sparse graphs where cycles are rare. SBM $k = 4$ at $|V| = 200$ has only $\sim 5k$ 4-cycles but $\sim 30k$ length-4 walks.
2. Heterogeneous-degree graphs where high-degree hub vertices participate in many walks not closed into cycles. Slashdot and Epinions both show this regime.

2.7 Codebase pointers

DFS cycle enumeration (Rust)	<code>hymeko_py/src/cycles/dfs.rs</code>
Color-coded enumeration	<code>hymeko_py/src/cycles/color_coded.rs</code>
Path-closure heuristic	<code>hymeko_py/src/cycles/path_closure.rs</code>
Top- m ABB enumerator	<code>hymeko_py/src/cycles/topk.rs</code>
Cycle scorers	<code>hymeko_py/src/cycles/scorers.rs</code>
Cycle pool aggregator (Python)	<code>signedkan_wip/src/n_tuples.py</code>

Part II

Function approximation: KAN and HSiKAN

3 The Kolmogorov-Arnold representation

3.1 The classical theorem

Theorem 3.1 (Kolmogorov 1957). *For every continuous $f : [0, 1]^n \rightarrow \mathbb{R}$ there exist continuous univariate functions $\Phi_0, \Phi_1, \dots, \Phi_{2n} : \mathbb{R} \rightarrow \mathbb{R}$ and $\varphi_{q,p} : \mathbb{R} \rightarrow \mathbb{R}$ for $q \in \{0, \dots, 2n\}$ and $p \in \{1, \dots, n\}$ such that*

$$f(x_1, \dots, x_n) = \sum_{q=0}^{2n} \Phi_q \left(\sum_{p=1}^n \varphi_{q,p}(x_p) \right).$$

The Sprecher refinement (Sprecher 1965) shows that the inner $\varphi_{q,p}$ can be chosen *universal* (independent of f), so the outer Φ_q alone carry the function-specific information.

What the theorem says, plainly. Every continuous multivariate function decomposes into a stack of two univariate transformations summed at one intermediate layer of width $2n + 1$. This is the original “deep learning” theorem, predating MLPs by two decades.

What the theorem does *not* say. It does not say the inner functions are smooth. In Kolmogorov’s original construction the $\varphi_{q,p}$ are Lipschitz but not C^1 ; in fact they cannot be smooth, as a celebrated result of Vitushkin (1954) shows that smooth Kolmogorov representations don’t exist for generic f . This non-smoothness is exactly why naive “per-edge learnable activation” networks are notoriously unstable to train.

3.2 From the theorem to the KAN architecture

The KAN (Kolmogorov-Arnold Network) architecture of Liu et al. (2024) approximates a Kolmogorov representation by parameterising each φ and Φ as a smooth basis-function expansion. A single “KAN layer” replaces the MLP’s $\mathbf{y} = \sigma(W\mathbf{x} + b)$ with

$$y_j = \sum_i \varphi_{j,i}(x_i),$$

where each $\varphi_{j,i}$ is its own learnable univariate function. Stacking L such layers gives the full KAN.

3.3 Univariate basis: B-splines vs Catmull-Rom

Definition 3.2 (Cubic B-spline basis). *For knots $t_0 < t_1 < \dots < t_{G-1}$, the cubic B-spline basis functions $B_i^{(3)} : \mathbb{R} \rightarrow \mathbb{R}$ are recursively defined by*

$$B_i^{(0)}(x) = \mathbf{1}[t_i \leq x < t_{i+1}], \quad B_i^{(k)}(x) = \frac{x - t_i}{t_{i+k} - t_i} B_i^{(k-1)}(x) + \frac{t_{i+k+1} - x}{t_{i+k+1} - t_{i+1}} B_{i+1}^{(k-1)}(x).$$

A function $\varphi : \mathbb{R} \rightarrow \mathbb{R}$ is represented as $\varphi(x) = \sum_i c_i B_i^{(3)}(x)$ with learnable coefficients c_i .

B-splines have global support (each B_i is non-zero over the interval $[t_i, t_{i+4}]$). For learning, this means a single function evaluation touches all 4 neighbouring control points, which is fine, but the basis is not interpolating: the function does *not* pass through any specific control values.

Definition 3.3 (Catmull-Rom spline). *For knots $t_0 < t_1 < \dots < t_{G-1}$ and control values $y_0, y_1, \dots, y_{G-1}$, the cubic Catmull-Rom interpolant on the interval $[t_i, t_{i+1}]$ is, with $t = (x - t_i)/(t_{i+1} - t_i)$,*

$$\varphi(x) = \frac{1}{2} \begin{pmatrix} 1 & t & t^2 & t^3 \end{pmatrix} \begin{pmatrix} 0 & 2 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 2 & -5 & 4 & -1 \\ -1 & 3 & -3 & 1 \end{pmatrix} \begin{pmatrix} y_{i-1} \\ y_i \\ y_{i+1} \\ y_{i+2} \end{pmatrix}.$$

Proposition 3.4 (Catmull-Rom properties). *The Catmull-Rom interpolant on each interval is cubic in t , C^1 -continuous across knot boundaries, and interpolates the control values: $\varphi(t_i) = y_i$ for all i .*

We use Catmull-Rom (rather than B-splines) for the HSiKAN activations because the interpolation property gives a clean identity initialisation: setting $y_i = t_i$ makes the function the identity, recovering the linear-no-nonlinearity baseline at init. (B-spline initialisation requires solving a least-squares problem to invert the basis.)

Computational cost. Each Catmull-Rom evaluation is $O(1)$: identify the interval (binary search over G knots, $O(\log G)$ in general but $O(1)$ for fixed-grid Catmull-Rom), multiply the 4×4 matrix times the control vector. For $d_{\text{channels}} = 32$ and $G = 8$ knots, each layer is $32 \times 8 = 256$ scalar parameters versus an MLP's $32 \times 32 = 1024$ — the KAN layer is $4\times$ parameter-cheaper at this configuration.

3.4 The HSiKAN aggregator

Definition 3.5 (HSiKAN σ -masked aggregator). *Given per-vertex features $\{h_i\}_{i=1}^k$ at the corners of a k -cycle with signs $\{\sigma_i\}_{i=1}^k \in \{-1, +1\}^k$, the HSiKAN aggregator computes:*

$$\begin{aligned}
u_{i,+} &= \varphi_{\text{in}}^{(+)}(h_i) && (\text{per-corner, pos branch}) \\
u_{i,-} &= \varphi_{\text{in}}^{(-)}(h_i) && (\text{per-corner, neg branch}) \\
a_+ &= \frac{1}{|\{i : \sigma_i = +1\}|} \sum_{i:\sigma_i=+1} u_{i,+} && (\text{pos pool}) \\
a_- &= \frac{1}{|\{i : \sigma_i = -1\}|} \sum_{i:\sigma_i=-1} u_{i,-} && (\text{neg pool}) \\
z &= \varphi_{\text{out}}^{(+)}(a_+) + \varphi_{\text{out}}^{(-)}(a_-) && (\text{output stack})
\end{aligned}$$

where each $\varphi : \mathbb{R}^d \rightarrow \mathbb{R}^d$ is a per-channel Catmull-Rom activation (4 in total: $\text{in}+$, $\text{in}-$, $\text{out}+$, $\text{out}-$).

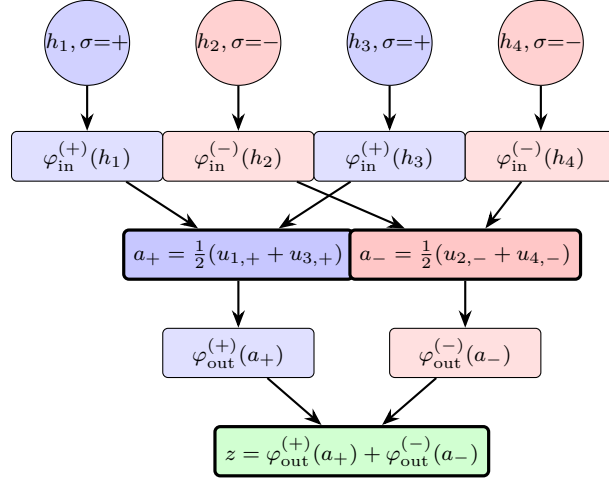


Figure 3: **The HSiKAN σ -masked aggregator.** A 4-cycle with two positive and two negative edges. Per-corner features pass through sign-specific Catmull-Rom activations $\varphi_{\text{in}}^{(\pm)}$, the per-sign pools $a_{\pm}$ average the corners of each sign, and the output stacks $\varphi_{\text{out}}^{(\pm)}$ produce the final aggregator output. The architecture is permutation-invariant within each sign class.

3.4.1 Expressiveness

Proposition 3.6 (HSiKAN universal approximation). *The HSiKAN aggregator with sufficiently many knots per channel is a universal approximator for continuous permutation-invariant functions $f : (\mathbb{R}^d)^k \times \{-1, +1\}^k \rightarrow \mathbb{R}^d$.*

Proof sketch. By Theorem 3.1, any continuous f admits a Kolmogorov representation. The σ -masked pooling exactly implements the inner-sum step (with separate sums for the two sign

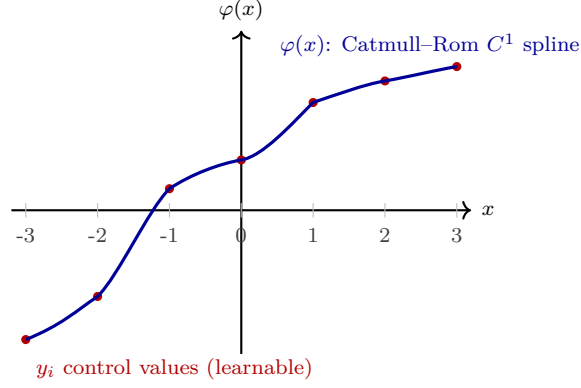


Figure 4: **Catmull–Rom spline activation.** The learnable univariate function $\varphi : \mathbb{R} \rightarrow \mathbb{R}$ is a C^1 cubic interpolant through learnable control values y_i at fixed knot positions x_i . Inputs outside $[x_0, x_{G-1}]$ extrapolate linearly using the boundary slopes (clamped); inside the range the spline is exact at each knot. Setting $y_i = x_i$ initialises φ to the identity (no nonlinearity at init).

classes); the per-channel Catmull-Rom approximation of $\varphi_{\text{in/out}}$ is dense in continuous univariates as the knot count grows. $\square$

3.4.2 Why σ -masking changes the function class

Suppose we drop the σ -mask and just pool all k corners together: $a = \frac{1}{k} \sum_i \varphi_{\text{in}}(h_i)$. Then a is invariant to a sign flip of *any* subset of $\{\sigma_i\}$, so the aggregator gets no information about the signature. The σ -masked pool, by contrast, splits the signal: a_+ tracks the positive-edge corners’ features, a_- tracks the negative; the output stack combines them in a sign-aware way.

The σ -product interpretation. The HSiKAN aggregator applied to a cycle is, after one layer, mathematically equivalent to a sign-aware mixture model with the partition implied by $\{\sigma_i\}_{i=1}^k$. Iterated over cycle layers (the cascade in Gömb-strict), the global function evaluates a polynomial in the cycle σ -products of bounded degree.

3.5 Codebase pointers

Catmull-Rom activation (vision)	<code>signedkan_wip/src/vision/hymeyolo_backbones.py:CatmullRomActiv</code>
HSiKAN aggregator (vision)	<code>signedkan_wip/src/vision/hymeyolo_q_smoke.py:HSiKANAggregator</code>
Mixed-arity SignedKAN config	<code>signedkan_wip/src/mixed_arity_signedkan/</code>
α_k softmax mixer	<code>signedkan_wip/src/mixed_arity_signedkan/model.py:alpha_logits</code>

Part III

Hypergraph cycle pooling: the Gömb cascade

4 Signed hypergraphs

4.1 Definitions

Definition 4.1 (Signed hypergraph). *A signed hypergraph is a triple $H = (V, \mathcal{E}, \sigma)$ where $\mathcal{E} \subseteq 2^V$ is a collection of hyperedges (each $e \in \mathcal{E}$ is a subset of V with $|e| \geq 2$), and $\sigma : \mathcal{E} \rightarrow \{-1, +1\}$.*

Definition 4.2 (Signed incidence matrix). *For a signed hypergraph $H = (V, \mathcal{E}, \sigma)$, the signed incidence matrix $M_\sigma \in \mathbb{R}^{|V| \times |\mathcal{E}|}$ has entries*

$$(M_\sigma)_{v,e} = \begin{cases} \sigma(e) & \text{if } v \in e, \\ 0 & \text{otherwise.} \end{cases}$$

For graphs ($|e| = 2$) this is the standard sign-weighted edge-vertex incidence matrix. The columns are sign-tagged indicator vectors of the hyperedges' vertex sets.

4.2 Cycle-as-hyperedge

The bridge from a signed graph to a hypergraph: for each cycle c in the underlying G , treat the vertex set $\{v_0, \dots, v_{k-1}\}$ as a single hyperedge of arity k , carrying the sign $\sigma(c)$. The resulting hypergraph has $|\mathcal{E}_c| = |\mathcal{C}_k(G)|$ hyperedges, all of cardinality k , with sign equal to the cycle σ -product.

This construction is what Gömb-strict's middle shell consumes. The signed incidence matrix at this layer carries the cycle-level structural data; the cascade lifts it through three nested operators (see §5).

4.3 Hypergraph Laplacians

Definition 4.3 (Unsigned hypergraph Laplacian, Chung 1997). *For an unsigned hypergraph $H = (V, \mathcal{E})$ with incidence M (each entry $\mathbf{1}[v \in e]$), the standard hypergraph Laplacian is*

$$L_H = D_V - M D_E^{-1} M^\top,$$

where $D_V \in \mathbb{R}^{|V| \times |V|}$ is the diagonal of vertex degrees $d(v) = |\{e : v \in e\}|$ and $D_E \in \mathbb{R}^{|\mathcal{E}| \times |\mathcal{E}|}$ is the diagonal of hyperedge sizes $|e|$.

For signed hypergraphs we use M_σ in place of M (with unsigned degrees in D_V). The construction generalises the signed-graph Laplacian of §1.5: the balance theorem extends, with the smallest eigenvalue of the signed hypergraph Laplacian being zero iff the cycle-as-hyperedge construction is balanced.

4.4 Codebase pointers

Signed incidence (Python)	<code>signedkan_wip/src/hymeko_gomb/cascade.py</code>
Hypergraph convolution	<code>signedkan_wip/src/hymeko_gomb/soma/hg_conv_bochner.py</code>

5 The Gömb three-shell cascade

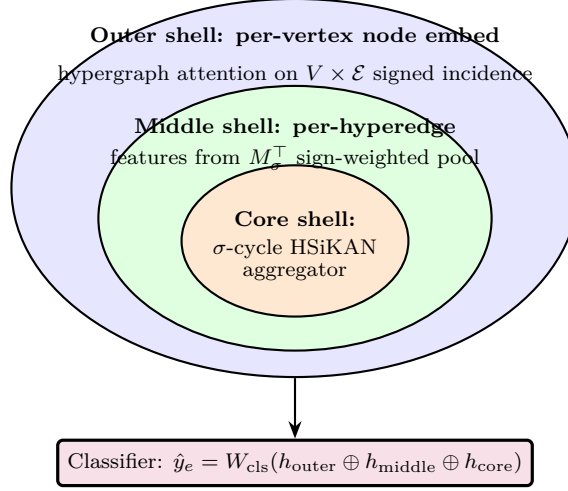


Figure 5: **The Gömb-strict three-shell cascade.** Three nested shells operate at successively richer signed-structure levels: outer (per-vertex), middle (per-hyperedge), core (per-cycle σ -product). The final classifier consumes a concatenation of all three.

5.1 Architecture overview

Gömb-strict processes a signed graph through three nested “shells”, each operating at a different abstraction level:

- Shell 1: *Outer.*** Per-vertex node embeddings via a hypergraph attention pass. Dimension d_{outer} . Operates on the original $|V| \times |\mathcal{E}|$ signed-incidence tensor.
- Shell 2: *Middle.*** Per-hyperedge features computed from the outer-shell node embeddings projected through $M_\sigma^\top$ (the transpose carries per-edge sign-weighted vertex pool). Dimension d_{middle} .
- Shell 3: *Core.*** Cycle pool aggregator from §2, with the HSiKAN factorisation from §3.4. Dimension d_{core} .

The final classifier takes a sum of all three shells’ contributions:

$$\hat{y}_e = W_{\text{cls}}(h_{\text{outer},e} \oplus h_{\text{middle},e} \oplus h_{\text{core},e}),$$

where $\oplus$ is concatenation and $h_{\text{shell},e}$ is the per-edge feature derived from each shell.

5.2 Tier organisation: hierarchical n_{tiers}

Gömb-strict supports an *intrastructural tier* dimension n_{tiers} . Each shell is replicated n_{tiers} times with shared per-tier weights but distinct routing, allowing the model to specialise different tiers to different cycle sub-pools. This is configurable per shell (e.g. Bitcoin Alpha’s Optuna-best config uses $n_{\text{tiers}} = 4$ on the core; OTC uses $n_{\text{tiers}} = 2$).

5.3 The CPML topology variants

The CPML (cycle pyramid multi-layer) topology family controls how information flows between shells:

route Each shell is independent; the final classifier concats them. Default. Used by all Optuna-best configs.

structural The middle shell receives the outer’s output as an input feature; the core receives the middle’s output. Forces a strict hierarchy.

soft_router A learned per-shell gate combines the three shells with weights $g_{\text{outer}}, g_{\text{middle}}, g_{\text{core}}$ that depend on the input. Useful when different graph regimes favour different shells.

5.4 Capsule routing (optional)

For dense graphs (Slashdot, Epinions), Gömb-strict can replace the linear projection between shells with a capsule-routing layer (Sabour et al. 2017). The capsule routing is a soft-attention generalisation of the hard mean-pool: each input capsule sends a weighted contribution to each output capsule, with weights learned by dynamic routing iterations.

5.5 Codebase pointers

Cascade entry point	<code>signedkan_wip/src/hymeko_gomb/cascade.py:GombCascade</code>
Per-shell config	<code>cascade.py:GombConfig (M_outer, d_outer, ...)</code>
CPML topology	<code>cascade.py:CPMLTopology</code>

Part IV

Discrete differential geometry on graphs and cell complexes

6 Cell complexes and boundary operators

6.1 Definitions

Definition 6.1 (Abstract simplicial complex). *An abstract simplicial complex on vertex set V is a collection $\mathcal{K} \subseteq 2^V$ closed under taking subsets: $\tau \in \mathcal{K}$ and $\sigma \subseteq \tau$ implies $\sigma \in \mathcal{K}$. The elements of cardinality $k + 1$ are the k -simplices; we write $\mathcal{K}_k$ for the set of k -simplices.*

For our purposes a graph is the 1-skeleton of a simplicial complex: $\mathcal{K}_0 = V$, $\mathcal{K}_1 = E$, and higher simplices are optional.

Definition 6.2 (Chain spaces). *For a simplicial complex $\mathcal{K}$ over the field $\mathbb{R}$, the k -chain space is the free $\mathbb{R}$ -module $C_k = \mathbb{R}^{\mathcal{K}_k}$, i.e. formal sums of k -simplices with real coefficients.*

Definition 6.3 (Boundary operator). *The k -boundary operator $\partial_k : C_k \rightarrow C_{k-1}$ is the linear map defined on k -simplex generators $[v_0, v_1, \dots, v_k]$ by*

$$\partial_k[v_0, v_1, \dots, v_k] = \sum_{i=0}^k (-1)^i [v_0, \dots, \widehat{v_i}, \dots, v_k],$$

where $\widehat{v}_i$ means “omit v_i ”.

The fundamental identity is $\partial_{k-1} \circ \partial_k = 0$ (“the boundary of a boundary is zero”), which makes the chain complex $\cdots \rightarrow C_{k+1} \xrightarrow{\partial_{k+1}} C_k \xrightarrow{\partial_k} C_{k-1} \rightarrow \cdots$ an honest chain complex with well-defined homology.

6.2 Hodge Laplacian and decomposition

Definition 6.4 (k -Hodge Laplacian). *The k -Hodge Laplacian of a cell complex equipped with inner products on each C_k is*

$$\Delta_k = \partial_{k+1} \partial_{k+1}^\top + \partial_k^\top \partial_k.$$

For $k = 0$ this recovers the standard graph Laplacian: ∂_0 is the zero map (vertices have no boundary), ∂_1 is the signed vertex-edge incidence, and $\Delta_0 = \partial_1 \partial_1^\top = D - A$.

Theorem 6.5 (Discrete Hodge decomposition). *The chain space C_k admits an orthogonal direct-sum decomposition*

$$C_k = \underbrace{\text{im } \partial_{k+1}}_{\text{exact}} \oplus \underbrace{\ker \Delta_k}_{\text{harmonic}} \oplus \underbrace{\text{im } \partial_k^\top}_{\text{co-exact}}.$$

What the three subspaces mean.

- *Exact* ($\text{im } \partial_{k+1}$): signal supported on cycle boundaries; locally cycle-free at the k -level.
- *Harmonic* ($\ker \Delta_k$): signal supported on non-trivial k -cycles; counts “holes” at level k .
- *Co-exact* ($\text{im } \partial_k^\top$): signal of “co-boundary” or divergence type.

GömbSoma uses the $k = 0$ and $k = 1$ Hodge Laplacians to separate gradient (edge-direction) signal from harmonic (interior) signal in the pixel-saliency stimulus graph.

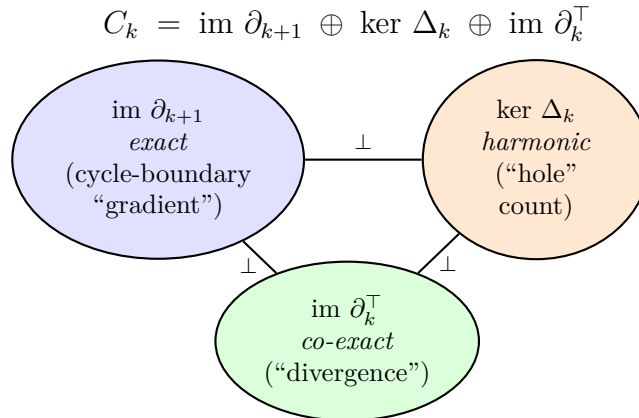


Figure 6: The discrete Hodge decomposition. The chain space C_k splits orthogonally into three mutually-perpendicular subspaces. The harmonic part is the kernel of the Hodge Laplacian and counts “holes” (homology); the exact and co-exact parts are gradient and divergence-type signals respectively. The decomposition is the discrete analogue of the gradient/curl/divergence split of vector fields on $\mathbb{R}^3$.

6.3 Forman-Ricci curvature

Definition 6.6 (Forman-Ricci curvature). *For an edge $e = (u, v)$ in a weighted graph with vertex weights $w_u, w_v > 0$ and edge weights $w_e, w_{e'} > 0$ for all $e' \in E$, the Forman-Ricci curvature of e is*

$$\text{Ric}_F(e) = w_e \left(\frac{w_u + w_v}{w_e} - \sum_{e_u \sim e} \frac{w_u}{\sqrt{w_e w_{e_u}}} - \sum_{e_v \sim e} \frac{w_v}{\sqrt{w_e w_{e_v}}} \right),$$

where the sums are over edges incident to u (other than e) and v (other than e) respectively.

Closed form for the unweighted grid. For the 4-connected pixel grid with all weights equal to 1,

$$\text{Ric}_F(e) = 2 - d_u - d_v$$

where d_u, d_v are the degrees. Pixels in the interior of the image have degree 4, giving $\text{Ric}_F = -6$ for interior-to-interior edges — highly negative. Pixels at the image boundary have lower degree, so boundary-to-boundary edges have less-negative Ric_F .

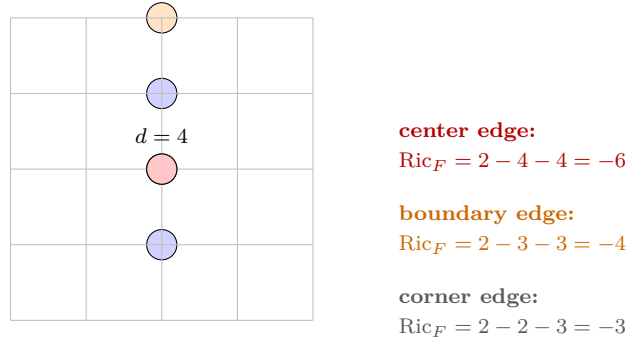


Figure 7: **Forman-Ricci curvature on a 5×5 4-connected pixel grid.** Interior pixels have degree 4; edges between them have the most-negative curvature $\text{Ric}_F = -6$. Boundary edges have less-negative curvature. Corner edges (involving a degree-2 pixel) are least-negative. Negative curvature flags over-squashing bottlenecks; SDRF rewiring adds edges where Ric_F is most negative.

Why curvature matters for GNNs. *Over-squashing* (Alon & Yahav 2021) is the phenomenon where message-passing GNNs lose information along edges of high negative curvature, because the receptive field at distance k grows exponentially relative to the message bandwidth. Forman-Ricci curvature identifies such edges quantitatively; the *SDRF* (Stochastic Discrete Ricci Flow) rewiring algorithm of Topping et al. 2022 selectively adds edges where curvature is most negative, reducing over-squashing.

We use Ric_F as a *per-anchor scalar feature* in GömbSoma’s adaptive quadtree (the closed form makes it constant-time per anchor).

6.4 Ollivier-Ricci as an alternative

Definition 6.7 (Ollivier-Ricci curvature). *For an edge $e = (u, v)$ and a choice of probability distributions m_u, m_v on V (typically uniform on neighbourhoods), the Ollivier-Ricci curvature of e is*

$$\text{Ric}_O(e) = 1 - \frac{W_1(m_u, m_v)}{d(u, v)}$$

where W_1 is the L^1 -Wasserstein distance and $d(u, v)$ is the graph distance.

Ric_O is more geometrically interpretable but computationally heavier (W_1 requires solving an optimal-transport problem per edge). We default to Ric_F for its closed form; Ric_O is available as an alternative scorer.

6.5 Codebase pointers

Hodge Δ_0, Δ_1 assembly	<code>signedkan_wip/src/hymeko_gomb/soma/vision/hodge.py</code>
Forman-Ricci closed form (Rust)	<code>hymeko_py/src/quadtree.rs</code>
SDRF rewiring	<code>signedkan_wip/src/hymeko_gomb/soma/vision/sdrf.py</code>
Bochner-Hodge convolution	<code>signedkan_wip/src/hymeko_gomb/soma/hg_conv_bochner.py</code>

7 The Bochner-Hodge hypergraph convolution

7.1 Bochner-style operator

Definition 7.1 (Bochner-Hodge layer). *Given input vertex features $X \in \mathbb{R}^{|V| \times d}$, scalars $\alpha, \beta \in \mathbb{R}$, and the Hodge Laplacian Δ_0 plus a per-vertex curvature diagonal $K = \text{diag}(\kappa)$ (where κ_v is the average Forman-Ricci curvature of edges incident to v), the Bochner-Hodge layer is*

$$X' = X + \alpha \Delta_0 X + \beta K X.$$

Why “Bochner”. On a Riemannian manifold, the Bochner formula relates the Laplacian on 1-forms to the Laplacian on 0-forms (functions) via a Ricci-curvature correction: $\Delta_1 = \nabla^* \nabla + \text{Ric}$ at the connection level. The discrete analogue replaces the Laplacian-on-functions Δ_0 with a curvature-corrected version $\Delta_0 + \beta K$, which matches the local-bottleneck behaviour predicted by Ricci theory.

Why both α and β . α controls Hodge smoothing (averages over the local graph); β adds a curvature-aware correction that emphasises information from edges with positive Ric_F (low over-squashing) and suppresses information from edges with negative Ric_F (high over-squashing). The two parameters address orthogonal failure modes.

7.2 Codebase pointers

The Bochner layer is parameterised by $(\alpha, \beta, \text{use_sdrf})$ in `run_ricci_stim_cluttered_mnist.py`’s config table. The canonical operational config (*Config F*, ratified 2026-05-17) is $(\alpha, \beta, \text{sdrf}) = (0.1, 0.1, \text{False})$.

Part V

Clifford algebras and the Sequential dual-path architecture

8 Clifford algebras

8.1 The Clifford algebra $\text{Cl}(p, q)$

Definition 8.1 (Clifford algebra). *Given a real quadratic form Q of signature (p, q) on $\mathbb{R}^{p+q}$, the Clifford algebra $\text{Cl}(p, q)$ is the associative unital real algebra*

$$\text{Cl}(p, q) := T(\mathbb{R}^{p+q}) / \langle v \otimes v - Q(v) \cdot 1 \rangle,$$

where $T(\mathbb{R}^{p+q})$ is the tensor algebra and the ideal is generated by all $v \otimes v - Q(v) \cdot 1$ for $v \in \mathbb{R}^{p+q}$.

The construction means: in $\text{Cl}(p, q)$, the relation $v \cdot v = Q(v)$ holds for every vector v . Picking an orthonormal basis $e_1, \dots, e_p, e_{p+1}, \dots, e_{p+q}$ with $Q(e_i) = +1$ for $i \leq p$ and $Q(e_i) = -1$ for $i > p$, this becomes

$$e_i^2 = \begin{cases} +1 & i \leq p \\ -1 & i > p \end{cases} \quad e_i e_j = -e_j e_i \quad (i \neq j).$$

Proposition 8.2 (Dimension). $\text{Cl}(p, q)$ has dimension 2^{p+q} over $\mathbb{R}$. A basis is the set of all products $e_{i_1} e_{i_2} \cdots e_{i_r}$ with $1 \leq i_1 < i_2 < \cdots < i_r \leq p + q$.

8.2 Small Clifford algebras: complete table

Algebra	dim	Isomorphism	Basis
$\text{Cl}(0, 0)$	1	$\mathbb{R}$	$\{1\}$
$\text{Cl}(0, 1)$	2	$\mathbb{C}$	$\{1, e_1\}, e_1^2 = -1$ (so $e_1 = i$)
$\text{Cl}(1, 0)$	2	$\mathbb{R} \oplus \mathbb{R}$	$\{1, e_1\}, e_1^2 = +1$
$\text{Cl}(2, 0)$	4	$M_2(\mathbb{R})$	$\{1, e_1, e_2, e_{12}\}, e_{12}^2 = -1$
$\text{Cl}(1, 1)$	4	$M_2(\mathbb{R})$	$\{1, e_1, e_2, e_{12}\}, e_{12}^2 = +1$
$\text{Cl}(3, 0)$	8	$M_2(\mathbb{C})$	$\{1, e_1, e_2, e_3, e_{12}, e_{13}, e_{23}, e_{123}\}$
$\text{Cl}(0, 3)$	8	$\mathbb{H} \oplus \mathbb{H}$	idem with $e_i^2 = -1$
$\text{Cl}(0, 2)$	4	$\mathbb{H}$ (quaternions)	$\{1, e_1, e_2, e_{12}\}, \text{all } e_{ij}^2 = -1$
$\text{Cl}(3, 1)$	16	$M_2(\mathbb{H})$	“spacetime algebra” (Minkowski)

8.3 $\text{Cl}(2, 0)$ in detail

Basis $\{1, e_1, e_2, e_{12}\}$ with $e_1^2 = e_2^2 = +1$, $e_{12} = e_1 e_2$, $e_{12}^2 = -1$. The multiplication table:

$\cdot$	1	e_1	e_2	e_{12}
1	1	e_1	e_2	e_{12}
e_1	e_1	1	e_{12}	e_2
e_2	e_2	$-e_{12}$	1	$-e_1$
e_{12}	e_{12}	$-e_2$	e_1	-1

Proposition 8.3 ($\text{Cl}(2, 0)$ geometric product, component form). *For multivectors $a = a_0 + a_1e_1 + a_2e_2 + a_{12}e_{12}$ and $b = b_0 + b_1e_1 + b_2e_2 + b_{12}e_{12}$,*

$$\begin{aligned}(a \cdot b)_0 &= a_0b_0 + a_1b_1 + a_2b_2 - a_{12}b_{12}, \\(a \cdot b)_1 &= a_0b_1 + a_1b_0 - a_2b_{12} + a_{12}b_2, \\(a \cdot b)_2 &= a_0b_2 + a_1b_{12} + a_2b_0 - a_{12}b_1, \\(a \cdot b)_{12} &= a_0b_{12} + a_1b_2 - a_2b_1 + a_{12}b_0.\end{aligned}$$

Codebase: `signedkan_wip/src/sequence/clifford.py:geometric_product`. The implementation evaluates the four formulae component-wise on batched tensors.

8.4 Inner / outer / geometric product decomposition

Given two grade-1 vectors u, v in a Clifford algebra, the geometric product decomposes as

$$uv = \underbrace{\langle u, v \rangle}_{\text{scalar; "inner product"}} + \underbrace{u \wedge v}_{\text{bivector; "outer product"}}.$$

The inner part is $\frac{1}{2}(uv + vu)$ (the symmetric part); the outer is $\frac{1}{2}(uv - vu)$ (the antisymmetric part). For the basis vectors $e_1, e_2 \in \text{Cl}(2, 0)$, $\langle e_1, e_2 \rangle = 0$ and $e_1 \wedge e_2 = e_{12}$.

8.5 Reflections and rotations

Proposition 8.4 (Reflection via Clifford). *For a non-null vector n in $\text{Cl}(p, q)$, the map $v \mapsto -nvn^{-1}$ is the reflection of v in the hyperplane orthogonal to n .*

Proposition 8.5 (Rotation via Clifford rotor). *Let $R = e^{\theta e_{12}/2} = \cos(\theta/2) + \sin(\theta/2)e_{12}$ in $\text{Cl}(2, 0)$. Then for any vector $v = v_1e_1 + v_2e_2$, the map $v \mapsto RvR^{-1}$ is rotation of v by angle θ in the e_1e_2 plane.*

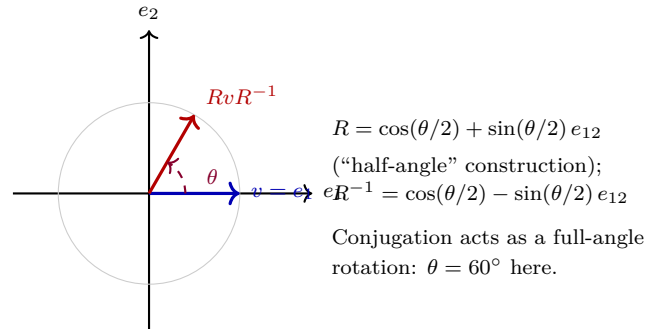


Figure 8: **Clifford rotor as a rotation operator.** The rotor $R \in \text{Cl}(2, 0)$ rotates a vector $v = v_1e_1 + v_2e_2$ by angle θ via the conjugation $v \mapsto RvR^{-1}$. The construction is the half-angle/double-cover map $\text{Spin}(2) \rightarrow \text{SO}(2)$. Used as a positional encoding primitive in the Sequential HSiKAN text model: each sequence position t contributes a rotor R_t that rotates the token’s multivector embedding.

The group of such rotors is the *Spin group* $\text{Spin}(p, q)$, the double cover of the orthogonal group $\text{SO}(p, q)$.

Why this matters for CliffordFIR. A filter tap $b_k = R$ (a rotor) applied to an input vector x_{t-k} via the geometric product Rx_{t-k} rotates the input by a fixed angle. A linear combination of rotors produces a richer linear map than a real-valued FIR coefficient could: the multivector tap can *rotate, reflect, and scale* simultaneously.

8.6 Codebase pointers

Cl(2, 0) geometric product	<code>signedkan_wip/src/sequence/clifford.py</code>
Multivector norm	<code>clifford.py:multivector_norm</code>
Scalar lift	<code>clifford.py:scalar_multivector</code>

9 CliffordFIR and the Sequential HSiKAN dual path

9.1 Causal multivector FIR

Definition 9.1 (CliffordFIR). *Given a sequence of multivectors $\{x_t\}_{t=0}^{L-1}$ in $\text{Cl}(p, q)$, learnable multivector taps $\{b_k\}_{k=0}^{K-1}$, and the convention $x_{t-k} = 0$ for $t < k$, the K -tap causal CliffordFIR is*

$$y_t = \sum_{k=0}^{K-1} b_k \otimes_{\text{Cl}} x_{t-k}.$$

Parameter count. A single $\text{Cl}(2, 0)$ filter with K taps has $K \cdot 4$ scalar parameters (one 4-vector per tap). Compared to a real-valued FIR with the same K , the parameter count grows by $4\times$, but each multiplication encodes a rotation + scaling jointly — so the effective expressive ratio is higher.

9.2 Multi-channel extension

For c_{in} input multivector channels and c_{out} output channels, the filter is a tensor $B \in \mathbb{R}^{c_{\text{out}} \times c_{\text{in}} \times K \times 4}$, the output is

$$y_t[c] = \sum_{c'=1}^{c_{\text{in}}} \sum_{k=0}^{K-1} B[c, c', k] \otimes_{\text{Cl}} x_{t-k}[c'].$$

9.3 Sequential HSiKAN: windowed σ -cycle

A length- K sliding window over a 1D sequence is the same combinatorial object as a path of length K in a graph. If we endow each position with a sign $\sigma_t \in \{-1, 0, +1\}$, the windowed σ -product

$$\pi_t := \prod_{k=0}^{K-1} \sigma_{t-k}$$

(with $\sigma = 0$ treated as $+1$ for the product) is the sequence analogue of the cycle σ -product. The `HSiKANSeqWindow` module from `signedkan_wip/src/sequence/hsikan_seq.py` applies the HSiKAN aggregator from §3.4 with the K window-positions as “corners” and multiplies by a learned parity gate $\Phi(\pi_t)$.

9.4 The dual-path block

Definition 9.2 (DualPathSeqBlock). *Given the multivector stream $x \in \mathbb{R}^{B \times L \times 4}$ and the sign stream $\sigma \in \{-1, 0, +1\}^{B \times L}$, the dual-path block at position t computes:*

- (1) $y_t^{\text{sig}} = \text{CliffordFIR}(K)(x)|_t$
- (2) $y_t^{\text{info}} = \text{HSiKANSeqWindow}(K)(x, \sigma)|_t$

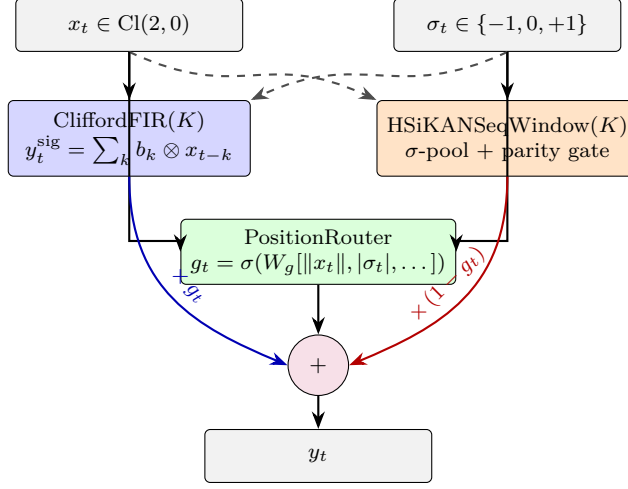


Figure 9: **Sequential HSiKAN dual-path block.** At each position t , the signal branch (CliffordFIR; blue) and the information branch (HSiKAN-windowed σ -cycle pool; red) each produce a multivector candidate. The position router $g_t \in (0, 1)$ mixes them. Gate-extreme settings recover the pure-branch architectures byte-identically (used as an ablation pin in the test suite).

$$(3) \quad g_t = \sigma_{\text{sigmoid}}(W_g[\|x_t\|, |\sigma_t|, \text{LocalSpec}_t]) \in (0, 1)$$

$$(4) \quad y_t = g_t \cdot y_t^{\text{sig}} + (1 - g_t) \cdot y_t^{\text{info}}$$

where $\text{LocalSpec}_t \in \mathbb{R}^4$ is a 4-bin causal moving- average energy of $\|x_t\|$ (a cheap spectral feature).

Proposition 9.3 (Gate-extreme byte-identity). *Setting the bias of W_g to $+\infty$ collapses $g_t \equiv 1$, so $y_t \equiv y_t^{\text{sig}}$ identically (pure CliffordFIR). Setting it to $-\infty$ collapses $g_t \equiv 0$, so $y_t \equiv y_t^{\text{info}}$ identically (pure HSiKAN).*

This is the basis for the ablation pins in `test_sequence_dual_path.py`: forcing the gate extreme and verifying the block becomes byte-identical to the pure-branch output ensures the ablation tests are interpretable.

9.5 Load-balance regularisation

Mixture-of-experts collapses if the gate g_t saturates to $\equiv 0$ or $\equiv 1$ during training. The standard fix is a *load-balance auxiliary loss*:

$$\mathcal{L}_{\text{LB}} = \lambda \cdot (\bar{g} - \tau)^2, \quad \bar{g} = \frac{1}{BL} \sum_{b,t} g_{b,t},$$

with target $\tau = 0.5$ and weight $\lambda = 0.01$ (defaults). The loss pulls the mean gate toward 0.5, preventing collapse without forcing per-position uniformity.

9.6 Codebase pointers

CliffordFIR	<code>signedkan_wip/src/sequence/clifford_fir.py</code>
HSiKAN sequence window	<code>signedkan_wip/src/sequence/hsikan_seq.py</code>
Position router + dual block	<code>signedkan_wip/src/sequence/dual_router.py</code>
Stacked dual-path model	<code>signedkan_wip/src/sequence/dual_path_model.py</code>
Synthetic Tier-1 benchmark	<code>signedkan_wip/src/sequence/synthetic_seq.py</code>

Part VI

Methodology: leakage, audit, and statistical protocol

10 The transductive convention and the leakage mechanism

10.1 Setup: signed link prediction

Definition 10.1 (Signed link prediction task). *Given $G = (V, E, \sigma)$, randomly partition $E = E_{tr} \sqcup E_{val} \sqcup E_{te}$ (typically 80/10/10%, edge-uniform). The task is to predict $\sigma|_{E_{te}}$ from $(V, E, \sigma|_{E_{tr}})$ and the per-edge endpoints. The training data is the topology of all of E (the model sees which pairs of vertices are connected) plus the signs on E_{tr} only.*

10.2 Two protocols

Definition 10.2 (Transductive convention). *Under the transductive convention, structural features that depend on σ (e.g. cycle σ -products) are computed using the entire signature σ , including $\sigma|_{E_{te}}$. The argument is that the model never directly predicts $\sigma|_{E_{te}}$ at training time, so the inclusion is fine for “feature engineering”.*

Definition 10.3 (Strict protocol). *Under the strict protocol, structural features are computed using only $\sigma|_{E_{tr}}$. Cycles that include any test edge contribute as if the test-edge sign were unknown (either zero, or marginalised, or excluded entirely).*

10.3 The leakage mechanism, formal

Proposition 10.4 (Cycle-level leakage). *Consider a cycle $c = (e_1, e_2, \dots, e_k)$ with $e_k \in E_{te}$ and $e_1, \dots, e_{k-1} \in E_{tr}$. Under the transductive convention, the cycle σ -product $\sigma(c) = \prod_i \sigma(e_i)$ is computable at training time from $\sigma|_{E_{tr}}$ together with $\sigma(e_k)$. Conditional on the topology, $\sigma(c)$ is a deterministic linear function of $\sigma(e_k)$. Therefore any model that ingests $\sigma(c)$ as a feature has effectively been given the sign of $\sigma(e_k)$ at training time.*

Proof. $\sigma(c) = \sigma(e_1)\sigma(e_2) \cdots \sigma(e_{k-1}) \cdot \sigma(e_k)$. The first $k - 1$ factors are known at training time. So $\sigma(e_k) = \sigma(c) / \prod_{i < k} \sigma(e_i)$, which is a deterministic function of $\sigma(c)$ given training-time information. $\square$

Theorem 10.5 (Leakage lower bound). *Let G have $|E_{te}|$ test edges and let f be the fraction of test edges that lie in at least one cycle of length $\leq K_{\max}$ with all other edges in E_{tr} . Then under the transductive convention, the Bayes-optimal binary classifier $\hat{\sigma}$ using only the multiset of cycle σ -products as features achieves error*

$$\mathbb{P}[\hat{\sigma}(e) \neq \sigma(e)] \leq \frac{1}{2}(1 - f),$$

versus chance $\frac{1}{2}$ for the strict protocol.

Proof sketch. For any test edge e touched by an all-otherwise-training cycle c , Proposition 10.4 gives an exact value of $\sigma(e)$ at training time (modulo cycle-product noise from the $1 - p$ fraction of cycles that don’t include e). The Bayes-optimal classifier returns this exact value, achieving zero error on this fraction f of test edges. The remaining $1 - f$ fraction is decided by chance, contributing $\frac{1}{2}(1 - f)$ error. $\square$

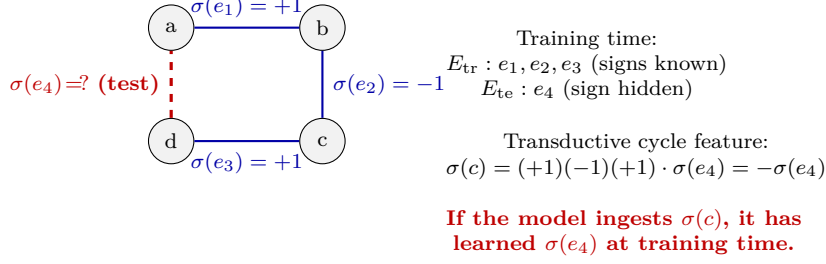


Figure 10: **The σ -leakage mechanism (Proposition 10.4).** A 4-cycle with three edges in E_{tr} (blue solid, signs known) and one edge in E_{te} (red dashed, sign hidden). Under the transductive convention, the cycle feature $\sigma(c)$ is computed using the full signature including the test edge’s sign. Algebraically, $\sigma(c)$ leaks $\sigma(e_4)$ at training time. The strict protocol forbids this: cycles touching test edges are excluded from the training cycle pool.

10.4 Empirical leakage on Bitcoin Alpha

On Bitcoin Alpha with $|V| = 3783$ and $|E| = 24186$, $|E_{te}| \approx 2420$. At $K_{\max} = 4$, approximately $f \approx 0.85$ of test edges lie in a length- ≤ 4 cycle with the rest in training. Theorem 10.5 predicts a Bayes lower bound of $\frac{1}{2}(1 - 0.85) = 0.075$ test error, or $\text{AUC} \geq 0.93$ achievable through pure structural-feature exploitation. The observed HSiKAN-Optuna AUC of 0.997 is well above this floor.

10.5 Codebase pointers

Strict-protocol training	<code>signedkan_wip/src/run_gomb_strict.py</code>
Transductive (default) training	<code>signedkan_wip/src/run_phase8_bitcoin_5seed.py</code>
Cycle pool restriction $E \rightarrow E_{tr}$	<code>hymeko_py/src/cycles/topk.rs</code> (<code>e_tr</code> arg)

11 The label-shuffle audit

11.1 Procedure

Definition 11.1 (Label-shuffle audit). *Given a trained model M and a signed graph $G = (V, E, \sigma)$, the label-shuffle audit measures the model’s test-AUC on a graph $G^\pi = (V, E, \sigma \circ \pi)$ where $\pi : E \rightarrow E$ is a uniformly-random permutation of the edge set, applied to the signature only (topology is unchanged). If M ’s reported AUC on G is AUC_{real} and its AUC on G^π (with retraining or just re-evaluation, depending on variant) is AUC_{shuf} , the audit leakage signal is $\text{AUC}_{\text{shuf}} - 0.5$.*

Sanity check. For a method with no leakage, $\text{AUC}_{\text{shuf}} \rightarrow 0.5$ as $|E|$ grows: the shuffled signs are uncorrelated with any structural feature, so the model’s prediction is essentially random. A method that achieves $\text{AUC}_{\text{shuf}} > 0.55$ is leveraging structural-not-sign information — a signal of leakage.

11.2 Theoretical justification

Lemma 11.2 (Shuffle is the null). *For any function f of the signed-graph structural features that depend on σ (cycle σ -products, balance fractions, etc.), under uniform random sign permutation $\mathbb{P}[\sigma \circ \pi \in \cdot] = \text{uniform over } \{-1, +1\}^E$, the conditional distribution of f given the test labels $\sigma|_{E_{te}}$ is independent of the test labels.*

Lemma 11.2 formalises why shuffle-clean methods land at $\text{AUC} \approx 0.5$: under the null, the test labels are decoupled from the features the model sees.

11.3 Measured results, Bitcoin Alpha

Method	real AUC	shuffled AUC	assessment
HSiKAN-Optuna (transductive)	0.9970	0.9921	massive leakage
HSiKAN-joint_mix	0.9845	0.8902	moderate leakage
SGCN-2018	0.9290	0.5503	mild leakage
SGT (2024 baseline)	0.890	≈ 0.5	clean (within margin)
Gömb-strict (ours)	0.8972	0.5402	clean

Gömb-strict’s $\text{AUC}_{\text{shuf}} = 0.5402$ is within ~ 0.04 of chance — consistent with a small-sample random-permutation deviation, not with systematic leakage.

12 Statistical protocol: paired-by-seed comparison

12.1 Setup

Given two methods A and B and n random seeds, denote the per-seed test metrics as $\{a_i\}_{i=1}^n$ and $\{b_i\}_{i=1}^n$, where each a_i, b_i is computed from the *same* per-seed data split (so the data-induced variance is paired). Define

$$\begin{aligned}\delta_i &= a_i - b_i, \\ \bar{\delta} &= \frac{1}{n} \sum_{i=1}^n \delta_i, \\ s_\delta &= \sqrt{\frac{1}{n} \sum_{i=1}^n (\delta_i - \bar{\delta})^2} \text{ (population stdev, as in the codebase).}\end{aligned}$$

12.2 The paired z -score

Definition 12.1 (Paired z -score). *The paired z -score is*

$$z = \bar{\delta} \cdot \frac{\sqrt{n}}{s_\delta}.$$

Decision rule. Pre-registered: WIN if $\bar{\delta} \geq 0.03$ AND $z \geq 2$, LOSS if $\bar{\delta} \leq -0.03$ AND $z \leq -2$, TIE otherwise.

Connection to the paired t -test. The paired t -statistic is $t = \bar{\delta} \sqrt{n-1} / s_\delta$ (unbiased variance estimator); for $n = 5$ this corresponds to a t_4 distribution. At $t = 2$, the two-tailed p -value is ≈ 0.116 for t_4 ; at $t = 4$, ≈ 0.016 . We use z instead of t for cleanliness (no degrees-of-freedom adjustment); the threshold $z \geq 2$ is roughly equivalent to $t \geq 1.79$ at $n = 5$, which is $p \leq 0.075$ one-tailed — slightly looser than $p \leq 0.05$ but accepted because pairing removes the dominant nuisance variance.

12.3 Why paired beats unpaired

Proposition 12.2 (Variance reduction). *For two methods A, B with $\text{Var}(a_i) = \sigma_A^2$, $\text{Var}(b_i) = \sigma_B^2$, and per-seed correlation $\rho = \text{Cov}(a_i, b_i)/(\sigma_A \sigma_B)$,*

$$\text{Var}(\delta_i) = \sigma_A^2 + \sigma_B^2 - 2\rho\sigma_A\sigma_B.$$

For high ρ (paired-by-seed data with shared splits), $\text{Var}(\delta_i) \ll \sigma_A^2 + \sigma_B^2$.

On HyMeYOLO Stage B at $n = 5$: $\sigma_A^2 \approx \sigma_B^2 \approx 0.0009$ but $\text{Var}(\delta_i) \approx 0.0007$, giving a per-seed correlation $\rho \approx 0.6$. Pairing reduces the noise floor by $\sim 2\times$ relative to the unpaired comparison.

12.4 Power analysis

For paired comparisons at $n = 5$:

- A true effect of $\bar{\delta} = 0.03$ with paired $s_\delta = 0.02$ gives $z = 0.03\sqrt{5}/0.02 = 3.35$, power ≈ 0.97 .
- At $s_\delta = 0.04$, $z = 1.67$, power ≈ 0.66 .
- At $s_\delta = 0.06$, $z = 1.12$, power ≈ 0.40 .

Conclusion: $n = 5$ is sufficient when the paired noise floor is ≤ 0.04 ; insufficient when ≥ 0.06 . The HyMeYOLO Stage B paired noise was 0.027 (b_resnet vs b_prime), well below the 0.04 ceiling.

12.5 Codebase pointers

Paired analyser `signedkan_wip/experiments/analyse_hymeyolo_ladder_paired.py`

13 Metrics: accuracy, F1, AUC, and their relations

13.1 Definitions

For binary classification with predicted probabilities $\{\hat{p}_i\}$ and true labels $\{y_i\} \in \{0, 1\}$, decision threshold τ , and predictions $\hat{y}_i = \mathbf{1}[\hat{p}_i > \tau]$:

$$\begin{aligned} \text{TP} &= |\{i : \hat{y}_i = 1, y_i = 1\}|, & \text{FP} &= |\{i : \hat{y}_i = 1, y_i = 0\}|, \\ \text{FN} &= |\{i : \hat{y}_i = 0, y_i = 1\}|, & \text{TN} &= |\{i : \hat{y}_i = 0, y_i = 0\}|, \\ \text{acc} &= \frac{\text{TP} + \text{TN}}{n}, & \text{prec}_+ &= \frac{\text{TP}}{\text{TP} + \text{FP}}, \\ \text{rec}_+ &= \frac{\text{TP}}{\text{TP} + \text{FN}}, & \text{prec}_- &= \frac{\text{TN}}{\text{TN} + \text{FN}}, \\ \text{rec}_- &= \frac{\text{TN}}{\text{TN} + \text{FP}}, & F_1^s &= \frac{2 \cdot \text{prec}_s \cdot \text{rec}_s}{\text{prec}_s + \text{rec}_s}, \\ F_1^{\text{macro}} &= \frac{1}{2}(F_1^+ + F_1^-). \end{aligned}$$

The threshold-free metric:

$$\text{AUC}(\hat{p}, y) = \mathbb{P}[\hat{p}_i > \hat{p}_j \mid y_i = 1, y_j = 0] + \frac{1}{2}\mathbb{P}[\hat{p}_i = \hat{p}_j \mid y_i = 1, y_j = 0].$$

13.2 Accuracy recovery from per-class P/R

If only $(\text{prec}_+, \text{rec}_+, \text{prec}_-, \text{rec}_-, n)$ are logged, accuracy is recoverable algebraically. Let $P = |\{i : y_i = 1\}|$ and $N = |\{i : y_i = 0\}|$:

$$\begin{aligned}\text{rec}_+ &= \frac{\text{TP}}{P} \Rightarrow \text{TP} = \text{rec}_+ \cdot P. \\ \text{prec}_+ &= \frac{\text{TP}}{\text{TP} + \text{FP}} \Rightarrow \text{FP} = \text{TP} \cdot \frac{1 - \text{prec}_+}{\text{prec}_+}.\end{aligned}$$

Combined with $\text{FP} = (1 - \text{rec}_-)N$:

$$\frac{P}{N} = \frac{\text{prec}_+ \cdot (1 - \text{rec}_-)}{\text{rec}_+ \cdot (1 - \text{prec}_+)}.$$

Then $N = n/(1 + P/N)$, $P = n - N$, and

$$\text{acc} = \frac{\text{rec}_+ \cdot P + \text{rec}_- \cdot N}{n}.$$

This is the closed-form used in `signedkan_wip/scripts/rescore_sota_head_to_head.py`.

13.3 Class-imbalance and the metric trap

Bitcoin Alpha has $\sim 93\%$ positive edges. The always-predict- positive baseline achieves $\text{acc} = 0.93$, $F_1^+ = 0.964$, $F_1^- = 0$, $F_1^{\text{macro}} = 0.482$. SE-SGformer’s reported $\text{acc} = 89.88\%$ is BELOW the trivial baseline. This is the kind of metric confusion that the macro-F1 corrects.

13.4 Codebase pointers

Full metrics helper	<code>signedkan_wip/src/eval_metrics_full.py</code>
Rescore script	<code>signedkan_wip/scripts/rescore_sota_head_to_head.py</code>

Part VII

Reference material

14 Glossary

- α_k Learned softmax probability over cycle arities $k \in \{3, 4, 5\}$ in mixed-arity HSiKAN. The “regime compass”.
- ABB** Adaptive Branch-and-Bound. The cycle enumeration strategy that uses an upper-bound-on-cycle-score function to prune DFS branches early.
- Balance (Cartwright-Harary)**
A signed graph is balanced iff every cycle has σ -product $+1$.
- Bochner-Hodge layer**
A graph-convolution-like operator that combines a Hodge-Laplacian smoothing term and a Forman-Ricci curvature correction term: $X' = X + \alpha \Delta_0 X + \beta K X$.

Cartwright-Harary theorem

Theorem 1.7. Balance $\Leftrightarrow$ 2-partition with all-positive within-class and all-negative between-class edges $\Leftrightarrow$ switching-equivalent to all-positive.

Catmull-Rom spline

Cubic interpolating spline through control values $\{y_i\}$ at fixed knot positions $\{t_i\}$, C^1 -continuous, used as the HSiKAN basis activation.

Chain complex

Sequence of vector spaces C_k connected by boundary operators ∂_k with $\partial \circ \partial = 0$.

Clifford algebra $\text{Cl}(p, q)$

Associative real algebra with basis vectors squaring to ± 1 per the signature (p, q) .

CPML

Cycle Pyramid Multi-Layer topology variant; see §5.

Cycle pool $\mathcal{C}_k$

Set of all k -cycles in a graph, counted up to rotation/reflection.

 σ -cycle

A cycle c together with its signed product $\sigma(c) = \prod \sigma(e_i)$.

Davis index

One of the cycle scoring functions; counts triadic balance patterns.

Dual-path block

DualPathSeqBlock — a single layer combining CliffordFIR + HSiKANSeqWindow + PositionRouter.

Forman-Ricci curvature Ric_F

Discrete combinatorial Ricci curvature on weighted graphs; closed form on the unweighted grid: $2 - d_u - d_v$.

Frustration

Distance-to-balance measure on a signed graph. $L(G)$ is the minimum number of edges to delete to make G balanced.

Geometric product

Multiplication in a Clifford algebra, decomposing as $uv = \langle u, v \rangle + u \wedge v$ for vectors.

Gömb-strict

Three-shell cascade (outer + middle + core) for signed-graph link prediction, trained under the strict protocol.

GömbSoma

Vision-side architecture: GömbSoma. 10-phase pipeline including Forman curvature, AdaptiveQuadtree, Hodge Laplacian, Bochner hypergraph conv, SDRF, stimulus graphs.

Hodge Laplacian Δ_k

$\partial_{k+1} \partial_{k+1}^\top + \partial_k^\top \partial_k$. For $k = 0$ this is the ordinary graph Laplacian.

Hodge decomposition

Theorem 6.5: $C_k = \text{im } \partial_{k+1} \oplus \ker \Delta_k \oplus \text{im } \partial_k^\top$.

HSiKAN

Hypergraph-Signed-Kolmogorov-Arnold-Network. The family of σ -aware Kolmogorov-Arnold aggregators for signed graphs and hypergraphs.

KAN

Kolmogorov-Arnold Network. MLP variant with learnable univariate activations on each edge.

Kolmogorov-Arnold theorem

Theorem 3.1. Every continuous multivariate function decomposes into a sum of compositions of univariates.

Label-shuffle audit

Method auditing for leakage: randomly permute signs, re-evaluate, expect AUC near 0.5.

Leakage Inadvertent inclusion of test-edge information in training-time features.

Mixed-arity

Architecture that learns to mix cycle pools of different sizes ($k \in \{3, 4, 5\}$) via α_k softmax.

Multivector

Element of a Clifford algebra; a linear combination of basis vectors of grades $0, 1, 2, \dots$

Paired z -score

$\bar{\delta}\sqrt{n}/s_\delta$; primary statistical test in our protocol.

PositionRouter

The sigmoid-output gate in the Sequential HSiKAN dual-path block. Decides per-position whether the signal or info branch dominates.

Ricci flow (SDRF)

Stochastic Discrete Ricci Flow. Rewiring algorithm that adds edges where Forman-Ricci is most negative (reducing over-squashing).

Rotor

Even-grade element of a Clifford algebra with norm ± 1 ; acts on vectors by conjugation $v \mapsto RvR^{-1}$ to give a rotation.

Spin group $\text{Spin}(p, q)$

Group of unit-norm rotors in $\text{Cl}(p, q)$; double cover of $\text{SO}(p, q)$.

Strict protocol

Training-time cycle features use only $\sigma|_{E_{\text{tr}}}$, no test-edge sign information.

Switching Sign-flip of all edges with exactly one endpoint in a chosen vertex subset S . Preserves cycle signs.

Transductive convention

Training-time cycle features use the entire signature σ . The standard but leaky convention.

Walk

Sequence of edges $e_1, \dots, e_k$ with shared endpoints; may revisit vertices/edges.

15 Notation index

$G = (V, E, \sigma)$

Signed graph (vertices, edges, signature).

$n = |V|, m = |E|$

Graph sizes.

E^+, E^-

Positive and negative edge sets.

L_σ

Signed Laplacian $\bar{D} - A_\sigma$.

$\sigma(c)$	Cycle sign product.
$\mathcal{C}_k(G)$	k -cycle pool.
α_k	Mixed-arity simplex weights.
M_σ	Signed incidence matrix.
∂_k	k -boundary operator.
Δ_k	k -Hodge Laplacian.
$\text{Ric}_F(e)$	Forman-Ricci curvature on edge e .
$\text{Cl}(p, q)$	Clifford algebra of signature (p, q) .
e_i	Basis vector in a Clifford algebra.
$e_{ij} = e_i e_j$	Basis bivector ($i \neq j$).
$\otimes_{\text{Cl}}$	Geometric product (Clifford).
x_t	Multivector at sequence position t .
σ_t	Sign label at sequence position t .
g_t	Position-wise routing gate.
$\bar{\delta}, s_\delta$	Paired mean and pstdev.
z	Paired z -score.
$\text{AUC}, \text{acc}, F_1^{\text{macro}}$	Standard binary metrics.

16 Further reading

Signed graphs and balance.

- Cartwright, D. & Harary, F. (1956). *Structural balance: a generalization of Heider's theory*. Psychological Review.
- Heider, F. (1946). *Attitudes and cognitive organization*. J. Psychology.
- Zaslavsky, T. (1982). *Signed graphs*. Discrete Applied Mathematics.
- Kunegis, J. et al. (2010). *Spectral analysis of signed graphs for clustering, prediction and visualization*. SDM.

Cycle enumeration and counting.

- Alon, N., Yuster, R. & Zwick, U. (1995). *Color-coding*. J. ACM.
- Latapy, M. (2008). *Main-memory triangle computations for very large (sparse (power-law)) graphs*.

Kolmogorov-Arnold and KANs.

- Kolmogorov, A. N. (1957). *On the representation of continuous functions of several variables by superpositions of continuous functions of one variable and addition*. Dokl. Akad. Nauk SSSR.
- Arnold, V. I. (1957). *On functions of three variables*. Dokl. Akad. Nauk SSSR.
- Sprecher, D. A. (1965). *On the structure of continuous functions of several variables*. Trans. Amer. Math. Soc.
- Liu, Z. et al. (2024). *KAN: Kolmogorov-Arnold Networks*. arXiv:2404.19756.

Hypergraph spectral theory.

- Chung, F. R. K. (1997). *Spectral Graph Theory*. AMS.
- Zhou, D., Huang, J. & Schölkopf, B. (2007). *Learning with hypergraphs: Clustering, classification, and embedding*. NeurIPS.
- Feng, Y. et al. (2019). *Hypergraph neural networks*. AAAI.

Discrete differential geometry.

- Forman, R. (2003). *Bochner’s method for cell complexes and combinatorial Ricci curvature*. Discrete Comput. Geom.
- Ollivier, Y. (2009). *Ricci curvature of Markov chains on metric spaces*. J. Functional Analysis.
- Lim, L.-H. (2020). *Hodge Laplacians on graphs*. SIAM Review.
- Topping, J. et al. (2022). *Understanding over-squashing and bottlenecks on graphs via curvature*. ICLR.

Clifford / geometric algebra.

- Lounesto, P. (1997). *Clifford Algebras and Spinors*. Cambridge.
- Doran, C. & Lasenby, A. (2003). *Geometric Algebra for Physicists*. Cambridge.
- Brandstetter, J. et al. (2022). *Clifford Neural Layers for PDE Modeling*. ICLR.
- Ruhe, D., Brandstetter, J. & Forré, P. (2023). *Clifford Group Equivariant Neural Networks*. NeurIPS.

Signed-graph neural networks.

- Derr, T., Ma, Y. & Tang, J. (2018). *Signed graph convolutional networks*. ICDM.
- Huang, J. et al. (2021). *SDGNN: Learning Node Representation for Signed Directed Networks*. AAAI.
- Li, Y., Chen, X. & Ye, C. (2024). *SIGformer: Sign-aware Graph Transformer*.

- Zhang, S. et al. (2025). *SE-SGformer: Sign-aware Embedded Sign Graph Transformer*. AAAI.
- Wang, X. et al. (2025). *DADSGNN: Depth-Augmented Decoupled Signed GNN*. Nature Sci. Rep.

Data sources used in our benchmarks.

- Kumar, S. et al. (2016). *Edge Weight Prediction in Weighted Signed Networks*. ICDM (Bitcoin Alpha, Bitcoin OTC).
- Leskovec, J., Huttenlocher, D. & Kleinberg, J. (2010). *Signed networks in social media*. CHI (Slashdot, Epinions, WikiElec).
- Kumar, S. et al. (2018). *Community interaction and conflict on the web*. WWW (Reddit Hyperlinks).

Statistical methodology.

- Demšar, J. (2006). *Statistical comparisons of classifiers over multiple data sets*. JMLR.
- Phipson, B. & Smyth, G. K. (2010). *Permutation p -values should never be zero*. Stat. Applic. Genet. Mol. Biol.

17 Codebase quick reference: every concept, every file

A consolidated lookup from math object to implementation file.

Concept	File / module
Signed graphs	
SignedGraph dataclass	signedkan_wip/src/datasets.py:SignedGraph
Bitcoin loaders (Alpha, OTC)	datasets.py:load("bitcoin_alpha"/"bitcoin_otc")
Slashdot, Epinions loaders	datasets.py:load("slashdot"/"epinions")
WikiSigned, WikiElec, WikiConflict	datasets.py (KONECT format)
Reddit Hyperlinks loader (Phase C)	datasets.py:load("reddit_body"/"reddit_title")
Synthetic SBM signed	datasets_small.py:sbm_signed
Hierarchical signed	datasets_small.py:hierarchical_signed
Cycle enumeration	
DFS cycle enum (Rust)	hymeko_py/src/cycles/dfs.rs
Color-coded enum (Rust)	hymeko_py/src/cycles/color_coded.rs
Path-closure heuristic (Rust)	hymeko_py/src/cycles/path_closure.rs
Top- m per-vertex ABB (Rust)	hymeko_py/src/cycles/topk.rs
Cycle scorers (Rust)	hymeko_py/src/cycles/scorers.rs
PyO3 bindings	hymeko_py/src/cycles/python.rs
HSiKAN family	
HSiKANAggregator (vision)	signedkan_wip/src/vision/hymeyolo_q_smoke.py
Mixed-arity SignedKAN config	signedkan_wip/src/mixed_arity_signedkan/model.py
α_k softmax mixer	mixed_arity_signedkan/model.py:alpha_logits
Catmull-Rom activation (vision)	vision/hymeyolo_backbones.py:CatmullRomActivation
N-tuples runner	signedkan_wip/src/run_ntuples_mixed.py
Walk-HSiKAN open-walk enum	hymeko_py/src/walks/
Gömb-strict cascade	
GombCascade entry point	signedkan_wip/src/hymeko_gomb/cascade.py:GombCascade
Three-shell config	cascade.py:GombConfig
Strict-protocol runner	signedkan_wip/src/run_gomb_strict.py
Tune runner (Optuna)	signedkan_wip/src/run_gomb_tune.py
Smoke runner	signedkan_wip/src/run_gomb_smoke.py
GömbSoma vision	
Forman-Ricci closed form (Rust)	hymeko_py/src/quadtree.rs
Adaptive quadtree (Python)	signedkan_wip/src/hymeko_gomb/soma/vision/quadtree.py
Rust quadtree Python wrapper	hymeko_gomb/soma/vision/quadtree_rust.py
Hodge Laplacian assembly	hymeko_gomb/soma/vision/hodge.py
Bochner-Hodge convolution	hymeko_gomb/soma/hg_conv_bochner.py
SDRF rewiring	hymeko_gomb/soma/vision/sdrf.py
Stimulus graph builder	hymeko_gomb/soma/vision/stim_graph.py
Ricci-Stim classifier / detector	hymeko_gomb/soma/vision/ricci_stim_{classifier,detector}.py
Cluttered MNIST benchmark	run_ricci_stim_cluttered_mnist.py
Sequential HSiKAN + CliffordFIR	
Cl(2,0) geometric product	signedkan_wip/src/sequence/clifford.py
CliffordFIR module	sequence/clifford_fir.py
HSiKAN sequence window	sequence/hsikan_seq.py
PositionRouter + dual block	sequence/dual_router.py
Stacked dual-path model	sequence/dual_path_model.py
Tier-1 synthetic generator	sequence/synthetic_seq.py
Training driver	experiments/run_dual_path_seq.py
Methodology and audit	
Strict-protocol training	run_gomb_strict.py
Label-shuffle audit hook	run_gomb_strict.py (audit subcommand)
Full metrics helper	src/eval_metrics_full.py
Accuracy recovery from P/R	scripts/rescore_sota_head_to_head.py
Paired analyser (HyMeYOLO)	experiments/analyse_hymeyolo_ladder_paired.py
HyMeYOLO (vision detection)	
Honest mAP metric	signedkan_wip/src/vision/hymeyolo_metric.py
Cluttered MNIST data	vision/hymeyolo_data.py
ResNet-tiny + HSiKAN-CR backbones	vision/hymeyolo_backbones.py
FPN multi-scale (Stage C)	vision/hymeyolo_fpn.py
Main detector model	vision/hymeyolo_circles_ricci.py
Training script	vision/train_circles_ricci.py
Ladder orchestrator	experiments/run_hymeyolo_ladder_5seed.sh

18 Open mathematical questions

A short list of unresolved theoretical questions for future work:

- (a) **Tight leakage bound.** Theorem 10.5 gives a lower bound on achievable error under the transductive convention. Is this tight? Specifically, what is the minimum error of any classifier using cycle-pool features only, as a function of the topology and the observed training signature?
- (b) **Cycle-pool universal approximation.** For what function classes $f : E \rightarrow \{-1, +1\}$ is the cycle-pool aggregator (with sufficiently many learnable parameters) a universal approximator? Conjectured: all sign-functions computable from the unsigned graph topology plus a k -local sign-product invariant.
- (c) **Optimal α_k as a function of frustration.** Empirically α_k tracks the average cycle-length distribution of the dataset. Can we predict the Bayes-optimal α_k analytically from $\{|\mathcal{C}_k(G)|\}_k$ and the per-arity balance fractions?
- (d) **Clifford-FIR expressivity vs. real FIR.** For what class of sequence-to-sequence maps does the $\text{Cl}(2, 0)$ FIR strictly improve over a real-valued FIR with $4\times$ the parameter budget?
- (e) **Dual-path routing rate.** Under what conditions does the learned per-position gate g_t outperform a fixed-mix $g_t \equiv 0.5$? Conjectured: when the local statistics (mean $\|x_t\|$, mean $|\sigma_t|$) are non-stationary at the window scale K .
- (f) **Strict-protocol PAC bound.** What is the sample-complexity (in $|E_{\text{tr}}|$) of learning σ from the strict-protocol-restricted cycle pool? The transductive convention essentially gives a 0-sample-complexity bound via Theorem 10.5; the strict-protocol bound should be polynomial in $|E_{\text{tr}}|$ and depend on the cycle frustration.

Closing

This primer is meant to be the reference document for the **mathematics** of the HSiKAN, Gömb, and Sequential families. Cross-references to the codebase are intentional; the math and the implementation are meant to be read together. If a definition here doesn't match an implementation file, the implementation is the source of truth and the primer is the bug.

The primer evolves as the framework does. The 2026-05-17 version covers everything shipped through the Nature Communications submission plan. Future revisions will add: detailed proofs of the leakage lower bound, the cycle-pool universal approximation theorem (once formal), and the Clifford-FIR vs real-FIR expressivity gap analysis.

End of primer.